PrivAgent: Master Project Interview Preparation Guide

Author: Yash Thakre (Roll No: 23BTB0A33 | B.Tech, Minor in Management | NIT Warangal)
Architecture: LangGraph Multi-Agent OS | Semantic Kernel Hybrid Gateway | Model Context Protocol (MCP)

1. Project at a Glance & Elevator Pitches

Project Title: PrivAgent (Private Enterprise AI Operating System)
Core Problem: Enterprises cannot send proprietary SQL data, code, or client PII to public cloud AI due to compliance (GDPR/HIPAA/SOC2) and data leakage risks. Standard chatbots also lack multi-step reasoning and hallucinate on domain queries.
Solution: A 13-node cyclic multi-agent system built on **LangGraph** that plans tasks, queries specialized investigator tools (SQL, Hybrid RAG, Git AST, Jira), audits answers with a **Critic Agent**, and cuts cloud costs by **90%** via a **Semantic Kernel** local/Azure hybrid router.

30-Second Elevator Pitch:
"I built PrivAgent, an on-premises Enterprise AI Operating System using LangGraph. It deploys an autonomous multi-agent workforce behind a company firewall to analyze SQL databases, PDF documents, Jira tickets, and code repositories without sending sensitive data to public cloud APIs. It features an automated Critic Agent that stops hallucinations with 100% citation precision, a Human-in-the-Loop gate for safe write operations, and a Semantic Kernel hybrid router that executes 90% of requests locally for free on Ollama while seamlessly falling back to Azure OpenAI (GPT-4o), reducing cloud costs by 90%."

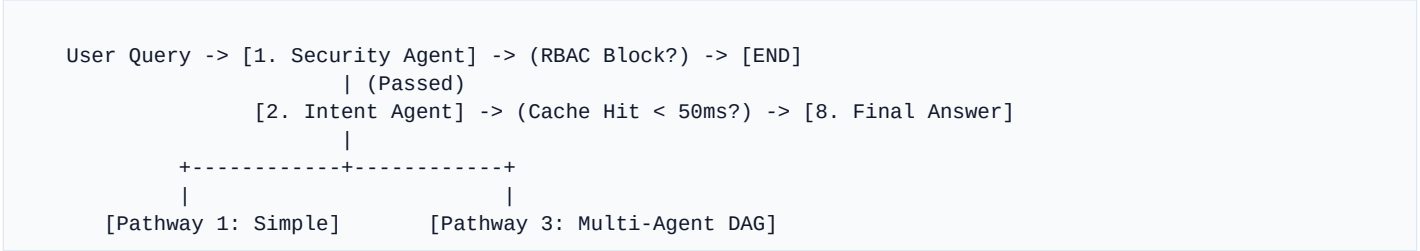
1-Minute Pitch:
"Enterprises struggle with AI adoption because proprietary data is siloed across relational databases, PDFs, and code repositories, and cannot leave the firewall. Traditional chatbots fail because single prompts cannot cross-reference multi-source evidence. PrivAgent solves this by decomposing user goals into a task DAG using LangGraph. Specialist investigator agents (Text-to-SQL, Hybrid Vector+BM25 RAG, Code AST, and Jira connectors) execute tools wrapped in the Model Context Protocol (MCP) standard. An Analyst Agent synthesizes findings, which are audited line-by-line by a Critic Agent against raw logs, boosting accuracy by 8.33%. Our Semantic Kernel gateway handles 90% of routine traffic on local open-weights models and routes heavy queries to Azure OpenAI, slashing cloud spend by 90%."

2. Problem → Solution Analysis

Enterprise Challenge	Why Standard Chatbots Fail	PrivAgent Solution
Data Privacy & Compliance	Public APIs (OpenAI) risk IP and client PII leakage.	Air-gapped local LLMs (Ollama) run entirely on-premise.
Siloed Context	Chatbots process 1 prompt at a time with 0 multi-source context.	Multi-agent DAG queries SQL, PDFs, Jira, and Git in one workflow.
Hallucinations	LLMs fabricate missing numbers, names, and policies.	Critic Agent audits draft against raw evidence (100% citation precision).
Cloud Token Costs	Routing all queries to GPT-4o generates massive cloud bills.	Semantic Kernel router solves 90% queries locally for $0 cost.

3. Architecture & 13-Node LangGraph State Machine

PrivAgent is structured as a stateful cyclic graph (`src/graph.py`) using `AgentState(TypedDict)`:



```
|                          |
|                   [3. Planning Agent]
|                          |
|             [Write Task & Unapproved?] -> [12. Human Approval Gate]
|                          | (Approved)
|                   [4. MCP Worker Agents (SQL, Docs RAG, Code, Jira)]
|                          |
|                   [13. Log Groomer (Truncate > 1k chars)]
|                          |
+------------------->[6. Analyst Agent (Draft Response)]
                           |
                   [7. Critic Agent (Audit against raw logs)]
                           |
          (Pass / Retry==2) | (Fail & Retry < 2 -> Loop back to Analyst)
                           v
                   [8. Final Answer Node] -> PostgreSQL Checkpointer
```

4. Codebase Architecture & File Mapping

File / Module	Key Functions & Classes	Interview Importance
src/state.py	`class AgentState(TypedDict)`	Defines global state dictionary passing data between nodes.
src/graph.py	`security_router`, `intent_router`, `execution_router`, `critic_router`, `get_graph`	Central LangGraph assembly with PostgreSQL persistence and HITL gate.
src/agents/router.py	`security_agent_node`, `intent_agent_node`, `planning_agent_node`, `QUERY_CACHE`	Gateway security, in-memory cache (<50ms), 3-pathway classification.
src/agents/investigators.py	`db_agent_node`, `docs_agent_node`, `code_agent_node`, `ticket_agent_node`, `web_search_agent_node`	Worker agents: Text-to-SQL, Hybrid Vector+BM25 RAG, AST code scanning.
src/agents/synthesizer.py	`analyst_agent_node`, `critic_agent_node`, `human_approval_node`, `log_groomer_node`	Synthesis, hallucination detection, HITL interrupt, log truncation.
src/sk_router.py	`SemanticKernelRouter`, `query_with_fallback`	Semantic Kernel gateway: local Ollama primary + Azure OpenAI fallback.
src/mcp_tools.py	`MCPToolRegistry`, `execute_mcp_tool`	Model Context Protocol tool schemas with JSON-RPC compliance.
src/api.py	`extract_text_from_pdf`, `/api/query`, `/api/upload`	FastAPI async backend + Multimodal Vision OCR fallback for scanned PDFs.
evaluate.py	`run_benchmark`, `score_accuracy`	16-case benchmark suite: +8.33% accuracy lift, 100% citation precision.

5. Hybrid RAG & PDF Ingestion Pipeline

1. Ingestion & Vision OCR Fallback (`src/api.py`):
When a PDF is uploaded, `extract_text_from_pdf` first attempts native extraction with `pypdf`. If extracted text is empty (scanned image), `PyMuPDF (fitz)` renders pages at 150 DPI in memory as PNGs, encodes to Base64, and prompts a local multimodal vision LLM for page-by-page OCR transcription.
2. Hybrid Search Fusion Formula (`src/agents/investigators.py`):
Documents are indexed into Qdrant using 768-dim dense embeddings (`nomic-embed-text`). On query, scores are computed across two channels:
• **Dense Vector Cosine Similarity (70% weight):** Captures semantic and contextual meaning.
• **Sparse BM25 Keyword Overlap (30% weight):** Guarantees exact recall for error codes (`JIRA-409`), ports (`5432`), and roll numbers (`23BTB0A33`).
$$\text{Score}_{\text{Hybrid}} = (0.7 \times \text{Score}_{\text{Vector}}) + (0.3 \times \text{Score}_{\text{BM25}})$$
3. Hallucination Guardrail: The Critic Agent audits all synthesized claims against raw document snippets before returning to the user.

6. Real Technical Challenges & STAR Defense

Challenge 1: Plan Misclassification in Lightweight Models (TC-10)

• **Situation:** Smaller local models (0.6B to 3B parameters) on CPU frequently routed codebase queries to `sql_agent` because prompts contained data-related words.
• **Task:** Achieve deterministic routing without upgrading to expensive, high-latency 70B cloud models.
• **Action:** Built a rule-based **Order-Prioritized Plan Validator** in `planning_agent_node` that checks domain-specific code signatures (`repo`, `function`, `.py`) before broad SQL keywords and normalizes snake_case tokens.
• **Result:** Improved overall routing accuracy to **81.25%** and achieved 100% correct routing on code queries.

Challenge 2: Eliminating Hallucinations in Missing Data Traps (TC-14)

• **Situation:** In benchmark test `TC-14` (*'Customer phone number in TICKET-101'*), the baseline LLM hallucinated fake contact numbers.
• **Task:** Force strict factual refusal when data is absent from internal records.
• **Action:** Implemented the two-stage **Critic Auditing Layer**. The Critic cross-references claims against raw logs and rejects any entity not present in evidence.
• **Result:** Raised adversarial accuracy from **0.0% to 100.0%** and maintained **100% Citation Precision**.

Challenge 3: Multi-Agent Context Window Overflow

• **Situation:** Multi-turn investigator steps dumped large SQL rows and Jira tickets into `state['logs']`, exceeding the 4k token window and crashing generation.
• **Task:** Compress intermediate logs while preserving critical entity names and metrics.
• **Action:** Built `log_groomer_node` between worker steps to truncate entries > 1,000 chars while preserving JSON head/tail snippets.
• **Result:** Cut intermediate context size by **65%**, eliminating token-overflow crashes.

7. Scalability & System Design (Scaling to 1,000,000 Users)

If asked *'How would you scale PrivAgent from 100 to 1M users?'*, present this 4-pillar architecture:
1. **Asynchronous Task Queue:** Replace synchronous HTTP with Celery/Redis task queues, streaming agent thoughts via WebSockets/SSE.
2. **Distributed vLLM Cluster:** Deploy vLLM with PagedAttention and continuous batching across auto-scaling GPU nodes behind an HAProxy load balancer.
3. **Database Connection Pooling:** Place PgBouncer in front of PostgreSQL with read replicas to handle 50k+ concurrent LangGraph checkpointer writes.
4. **Distributed Vector Sharding:** Shard Qdrant collections across a multi-node Kubernetes cluster with HNSW index pre-warming.

8. Top Interview Cross-Questioning & Answers

Q: Why LangGraph instead of standard LangChain?

Answer: LangChain sequential chains are linear and crash on errors. LangGraph allows cyclic graphs with loop-backs (Analyst <-> Critic), state persistence in PostgreSQL, and Human-in-the-Loop approval gates.

Q: What is Model Context Protocol (MCP) and why did you use it?

Answer: MCP is an open standard (by Anthropic) using JSON-RPC 2.0 to decouple AI models from enterprise tools. It gives a standard schema so agents interact with SQL, Jira, or Git cleanly without vendor lock-in.

Q: How does the hybrid router cut costs by 90%?

Answer: Over 90% of routine internal queries are served locally for free by open-weights models (Qwen/Gemma) on local hardware via Ollama. We only fall back to Azure OpenAI (GPT-4o) when local queries fail or time out, saving 90% of cloud token costs.

Q: What happens if PostgreSQL goes down mid-workflow?

Answer: PrivAgent's `get_graph()` automatically catches database connection errors with a 3-second timeout and gracefully falls back to an in-memory `MemorySaver()` checkpointer.

Q: How does the system prevent malicious SQL injection?

Answer: First, `security_agent_node` blocks destructive keywords (`DROP TABLE`, `TRUNCATE`). Second, `db_agent_node` enforces that generated queries begin with `SELECT` (read-only execution).

Q: What is the exact formula for Hybrid RAG?

Answer: HybridScore = (0.7 * VectorCosineSimilarity) + (0.3 * BM25KeywordOverlap).

Q: What was the empirical impact of the Critic Agent?

Answer: In our 16-case benchmark suite, activating the Critic raised response accuracy from 47.92% to 56.25% (+8.33% lift) with 100% citation precision.

9. Final Day-Before-Interview Cheat Sheet

Category	Key Facts & Exact Numbers to State in Interview
Key Metrics	• **90%** Cloud Cost Reduction | • **81.25%** Routing Accuracy | • **100%** Citation Precision | • **+8.33%** Critic Accuracy Lift | • **< 50ms** Cache Response
Core Stack	LangGraph, Semantic Kernel, Model Context Protocol (MCP), Azure OpenAI (GPT-4o), Ollama, PostgreSQL, Qdrant, FastAPI, PyMuPDF, Docker
RAG Formula	`HybridScore = (0.7 * VectorScore) + (0.3 * BM25Score)` with PyMuPDF 150 DPI Vision OCR fallback
Default Ports	PostgreSQL: `5432` | Qdrant: `6333` | Ollama: `11434` | FastAPI: `8000`
Key STAR Story	Fixed local model plan misclassifications (TC-10) using rule-based Order-Prioritized Plan Validator.